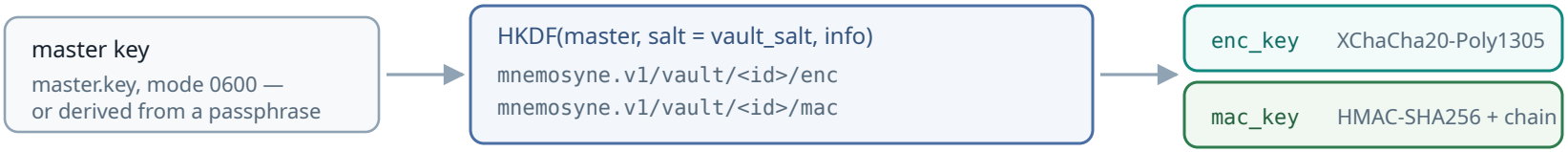
One master key, a separate key per vault, and an AAD that names the row

Isolation is cryptographic, not a WHERE clause: a blob lifted from one vault into another does not decrypt, because the vault id is authenticated.

Derivation



The two keys are independent outputs of the same derivation, so an integrity tag can never be forged from a decryption key and the two roles stay separate.

Both live in a zeroize-on-drop wrapper, are never Debug-printed, and the key file is fsynced at creation — a key that is not durable is a vault that cannot be opened again after a power loss.

A vault carries a `keycheck` marker, so a wrong passphrase is refused up front rather than surfacing later as a wall of decryption failures.

What every sealed blob authenticates alongside itself

The AEAD's associated data is not decoration — it is what makes a ciphertext meaningful only in the exact position it was written.

AAD = vault id + record id + domain

`<drawer id>`	content
`<drawer id>/emb`	embedding
`<drawer id>/tok`	ColBERT token matrix
`<drawer id>/pq`	PQ codes, codebook, pages

so all of these FAIL, not merely return nothing

A blob copied from vault A into vault B — the id differs.

A row's ciphertext moved onto another row — the record id differs.

An embedding swapped in where content was expected — the domain suffix differs.

Rotation — a new key, the same history

Every artifact is decrypted under the old key and re-sealed under the new one in a single transaction, and the chain is re-keyed over the audit bytes it already had — the history is preserved, not restarted.

The manifest is staged as `vault.json.next` and swapped, so a crash mid-rotation leaves one of two consistent states and is reconciled at the next open.

Sharing — an export bundle for one recipient

X25519 ephemeral-static to the recipient's public key, through HKDF, into XChaCha20-Poly1305. The sender's vault keys never leave the machine and the bundle is readable by exactly one holder of the matching private key.

This is the no-lock-in path: a vault can be handed over whole without ever exposing it to the operator moving it.

Where the boundary honestly sits

Sealing protects data at rest. A running process holds the derived keys in memory, so an attacker with live access to the process, or an operator hosting the engine, is inside the boundary — no at-rest scheme changes that, and claiming otherwise would be dishonest.

What it does buy, precisely: a stolen disk, a leaked backup, a snapshot, a decommissioned drive, or a copied database file yields ciphertext and the metadata inventory — never the words.

The remote index tier is treated as untrusted on the same reasoning: only sealed payloads are pushed to it, and anything it returns is re-verified locally against the vault's own HMAC before a single byte reaches the caller. An accelerator is not a source of truth.